HASSHのホワイトリストを用いたFPGAベースのSSHブルートフォース対策NIPS

久住 洸平[1,a)]　小林 良太郎[1,b)]

概要：近年，クラウドサービスやサーバの遠隔管理において，Secure Shell (SSH) が広く利用されている．一方，SSH サービスは外部ネットワークに公開される場合が多く，弱いパスワードや漏えいした認証情報を利用したブルートフォース攻撃の対象となる．従来の対策では，送信元 Internet Protocol (IP) アドレスごとの認証失敗回数に基づいて攻撃を検知する方式が広く用いられている．しかし，複数の送信元 IP アドレスから認証試行を分散させる攻撃では，IP アドレスごとの試行回数が閾値に達せず，検知を回避されやすい．そこで本研究では，SSH_MSG_KEXINIT から算出した HASSH によりクライアントを識別し，ホワイトリストに掲載されていない未知のクライアントからの接続を Transmission Control Protocol (TCP) の Reset (RST) パケットの注入によって遮断する Network-based Intrusion Prevention System (NIPS) を提案する．HASSH は同一のクライアント実装であれば送信元 IP アドレスに依存せず一致するため，IP アドレスを分散させる攻撃に対しても実装単位での識別が可能となる．さらに，これらの処理を Field Programmable Gate Array (FPGA) 上に実装することで，フレーム到達から遮断までを回線内で完結させ，ホストへの問い合わせを介さない低遅延な遮断を実現する．評価では，ホワイトリスト非掲載のクライアントに対する遮断動作と，掲載クライアントの通信への影響を実機上で確認する．

キーワード：FPGA，ブルートフォース攻撃，実装

An FPGA-Based NIPS against SSH Brute-Force Attacks Using a HASSH Whitelist

Kohei Kusumi[1,a)]　Ryotaro Kobayashi[1,b)]

Abstract: In recent years, Secure Shell (SSH) has been widely used in cloud services and remote server management. On the other hand, SSH services are often exposed to external networks, and become targets of brute-force attacks that exploit weak passwords or leaked credentials. In conventional countermeasures, a method that detects attacks based on the number of authentication failures per source Internet Protocol (IP) address is widely used. However, in attacks that distribute authentication attempts across multiple source IP addresses, the number of attempts per IP address does not reach the threshold, and detection is easily evaded. Therefore, in this study, we propose a Network-based Intrusion Prevention System (NIPS) that identifies clients using the HASSH calculated from SSH_MSG_KEXINIT and blocks connections from unknown clients not listed in a whitelist by injecting a Transmission Control Protocol (TCP) reset (RST) packet. Because HASSH matches regardless of the source IP address as long as the client implementation is the same, identification is possible even against attacks that distribute IP addresses. Furthermore, by implementing these processes on a Field Programmable Gate Array (FPGA), the path from packet arrival to blocking is completed within the FPGA, achieving low-latency blocking that does not involve any query to the host. In the evaluation, we verify on real hardware the blocking behavior against clients not listed in the whitelist and the impact on the communication of listed clients.

Keywords: FPGA, Brute-Force Attack, Implementation

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. 背景

近年，Internet of Things (IoT) 機器やサーバの遠隔操作において，Secure Shell (SSH) は重要な通信基盤となっている．SSH は暗号化された安全な遠隔操作を提供する一方で，外部ネットワークに公開される場合も多く，弱いパスワードや漏えいした認証情報を用いた辞書攻撃および総当たり攻撃の対象となりやすい．情報通信研究機構が公開している Network Incident analysis Center for Tactical Emergency Response (NICTER) の観測レポート 2025 によれば，SSH で使用される Transmission Control Protocol (TCP) の 22 番ポート宛の通信が宛先ポート別の上位に観測され，SSH サービスを狙った攻撃が前年に引き続き多く確認された [1]．本研究においては，サイバー攻撃の中でも特に SSH に対するブルートフォース攻撃に焦点を当てる．ブルートフォース攻撃は，特定の識別子に対してすべての文字列や数値の組み合わせを総当たりし，不正ログインを試みる攻撃手法である．SSH に対する既存対策としては，ログ監視に基づく検知や，Internet Protocol (IP) アドレスに基づく監視が用いられている．しかし，送信元が分散する攻撃では IP アドレス単位の対応に限界がある．

そこで本研究では，SSH 接続時に現れるクライアント実装の特徴に着目し，通常利用される SSH クライアントと攻撃ツール由来のクライアントとの差異を観測する．SSH の暗号化前ハンドシェイクで送信される SSH_MSG_KEXINIT には，クライアント実装に依存する特徴が含まれる．本研究では，この情報から特徴量である HASSH を抽出する機構を Field Programmable Gate Array (FPGA) 上に実装する．HASSH は，SSH 接続時に交換される SSH_MSG_KEXINIT に含まれる鍵交換方式，暗号方式，Message Authentication Code (MAC) 方式，圧縮方式のアルゴリズムリストを用いて，SSH クライアントの実装差を指紋化する手法である [2]．HASSH では，これらのリストから生成した文字列を Message Digest 5 (MD5) ハッシュ値として表現することで，SSH クライアントごとの特徴を固定長の識別子として扱う．HASSH は同一のクライアント実装であれば送信元 IP アドレスに依存せず同じ値になるため，IP アドレスを分散させる攻撃であっても実装単位で識別できる．そこで本研究では，あらかじめ許可するクライアントの HASSH をホワイトリストとして登録し，掲載されていないクライアントからの接続を遮断する方式をとる．この方式は認証失敗回数を集計しないため，試行が何台の端末に分散されても判定が変わらない．また，ソフトウェアの実装では高トラフィック時の遅延や Central Processing Unit (CPU) 負荷が課題となる．FPGA は通信経路上でパケット処理をハードウェア化でき，インラインに配置することでホスト側にソフトウェアを導入せずに通信を観測できるため，本研究では FPGA を使用する．本研究では，FPGA を用いて通信経路上で KEXINIT から HASSH を算出し，ホワイトリストに掲載されていないクライアントを遮断する Network-based Intrusion Prevention System (NIPS) を実装し，評価する．

本論文の構成は以下の通りである．まず，第 2 章では関連研究について述べる．第 3 章では基礎技術，第 4 章では提案システム，第 5 章では提案システムの検証方法について述べる．第 6 章では評価結果を述べ，第 7 章では考察し，第 8 章で本論文をまとめる．

2. 関連研究

本論文では，SSH_MSG_KEXINIT からブルートフォース攻撃ツールを識別し，FPGA 上で遮断する NIPS を提案する．そこで FPGA を用いた Network-based Intrusion Detection System (NIDS) や NIPS を実装する既存研究がどのように行われているかを述べる．また，SSH_MSG_KEXINIT を用いた NIDS や NIPS に関する既存研究についても述べる．

2.1 FPGA を使った既存研究

FPGA を用いて NIDS や NIPS の処理を高速化する研究は，これまでに複数報告されている．

Zhao らは，TCP ストリーム再構成や文字列照合の大部分を FPGA 上で実行する Pigasus を提案し，1 個の FPGA と平均 5 個の CPU コアで 100 Gbps の処理を実現した [3]．また，Jaic らは，高負荷状態においてソフトウェアで実装した NIDS では 90%近いパケット損失が発生するのに対し，FPGA では 10 Gbps の入力に対してパケット損失が発生しなかったと報告した [4]．これらの研究から，FPGA 上でのパケット処理が高速に行えることがわかる．一方，これらは SSH ブルートフォース攻撃に特化した検知を対象としておらず，本研究が着目する低遅延な遮断も対象としていない．

2.2 SSH ブルートフォース攻撃に関する既存研究

従来の分散型 SSH ブルートフォース対策では，fail2ban などの送信元 IP アドレスごとの認証失敗回数に基づく検知が広く用いられる．しかし，複数の送信元 IP アドレスに認証試行を分散させる攻撃では，各 IP アドレスの試行回数が閾値に達しない可能性がある．

Javed らは，多数の送信元 IP アドレスから低頻度で認証試行を行う分散 SSH ブルートフォース攻撃を対象とし，複数の接続に共通する認証試行の特徴を分析することで，IP アドレス単位の監視では検知が困難な攻撃を検出する手法を提案した [5]．同手法では，SSH サーバの認証ログから算

[1] 工学院大学
Kogakuin University, Shinjuku, Tokyo 163-8677, Japan
[a)] j123100@ns.kogakuin.ac.jp
[b)] ryo.kobayashi@cc.kogakuin.ac.jp

© 2026 Information Processing Society of Japan

出した認証失敗回数に対し，Cumulative Sum (CUSUM) による変化点検知を適用してサイト全体における認証失敗の増加を検出し，複数の送信元が共通して試行したユーザ名や SSH サーバから分散攻撃の送信元を特定する．これに対し本研究では，認証ログを参照せず，通信経路上で観測した SSH_MSG_KEXINIT のみからクライアントを識別する．

Ghiëtte らは，SSH クライアントのバージョン文字列と，SSH_MSG_KEXINIT に含まれる各アルゴリズムの種類と順序から独自の指紋を生成し，ブルートフォース攻撃に使用されるクライアントツールを識別する手法を提案した [6]．また，同一の指紋を持つ異なる送信元 IP アドレスの活動を関連付けることで，低速攻撃や分散攻撃を分析できることを示した．一方，同研究は攻撃ツールの識別および脅威分析を目的としており，指紋に基づく接続の遮断や，FPGA 上での実装については検討していない．

Baiardi らは，SSH_MSG_KEXINIT から算出した HASSH を，あらかじめ収集した悪性 HASSH のデータベースと照合し，悪性 SSH クライアントをハンドシェイク段階で遮断する Kex-Filtering を提案した [7]．この手法は，既知の悪性 HASSH を持つ接続を認証処理の前に遮断できる一方，ブラックリストに登録されていない未知の HASSH を用いる攻撃には対応しにくい．これに対し本研究では，悪性 HASSH のブラックリストではなく，許可するクライアントのホワイトリストを用いる．この方式の利点と限界については第 7.1 節で述べる．

3. 基礎技術

本研究の実装は，GitHub 上で公開されている c10gx_icmp_server を土台としている [8]．これは Intel Cyclone 10 GX FPGA 開発キット向けに書かれた Internet Control Message Protocol (ICMP) 応答サーバであり，10 Gigabit Ethernet の Media Access Control (MAC) 層，Physical Layer (PHY)，クロック生成回路，リセット回路を含む．なお，本論文では，Ethernet の Media Access Control 層を MAC 層, 暗号方式の Message Authentication Code を MAC 方式と表記して区別する．

本研究では，このうち MAC 層と PHY，クロック生成回路，リセット回路を利用し，ICMP および Address Resolution Protocol (ARP) の応答部を提案システムの監視パイプラインに置き換えた．

4. 提案システム

本章では，従来の送信元 IP アドレス単位の SSH ブルートフォース攻撃検知方式と，本研究で提案する HASSH のホワイトリストによる方式を比較しながら述べる．

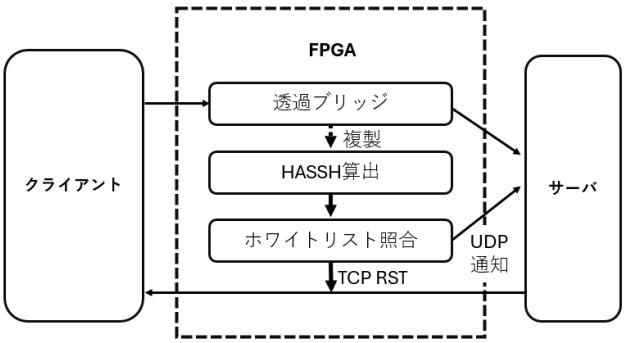


図 1 HASSH のホワイトリストによる提案方式

4.1 2 つの検知方式の概要

4.1.1 送信元 IP アドレス単位の検知方式

従来方式では，一定時間内に同一の送信元 IP アドレスから発生した認証失敗回数を集計し，その回数が設定した閾値を超えた場合に，SSH ブルートフォース攻撃として検知する．単一の送信元から多数の認証試行が行われる場合には検知できるが，複数の送信元 IP アドレスに認証試行を分散させる攻撃では，各 IP アドレスの認証失敗回数が閾値未満となり，検知を回避される可能性が高い．

4.1.2 提案システム

本研究で提案する SSH ブルートフォース攻撃検知システムの概要を図 1 に示す．図 1 の破線の枠が FPGA であり，枠内の透過ブリッジが本線，そこから分岐した矢印が解析系への複製を表す．複製は HASSH 算出とホワイトリスト照合を順に通り，照合結果に応じてサーバ側へ UDP 通知が，クライアント側へ TCP RST が送出される．提案システムは，クライアントと SSH サーバ間に配置した FPGA 上で，双方向の SSH 通信を転送しながら監視する．各 SSH 接続について，SSH_MSG_KEXINIT から HASSH を算出し，ホワイトリストと照合する．提案方式は接続ごとに独立して判定し，認証失敗回数を集計しない．そのため，複数の送信元 IP アドレスに認証試行が分散されても各接続の判定は変わらず，従来の IP アドレス単位の方式で問題となる閾値の回避が成立しない．

提案システムは，解析の完了を待たずに通信を転送する．解析は KEXINIT の複製に対して行うため，本線の転送は解析の進行状況に影響されない．解析が完了し，ホワイトリストに一致しないと判定された場合にのみ，クライアントへ Reset (RST) フラグを設定した TCP パケット，すなわち TCP RST を送出して接続を切断する．KEXINIT は鍵交換の最初のメッセージであり，認証はその後に行われる．したがって解析が鍵交換の完了より早ければ，認証試行の前に接続を切断できる．

4.2 提案システムの実装

本節では，FPGA 上に実装する提案 NIPS の構成および処理について述べる．提案システムの実装構成を図 2 に示

© 2026 Information Processing Society of Japan

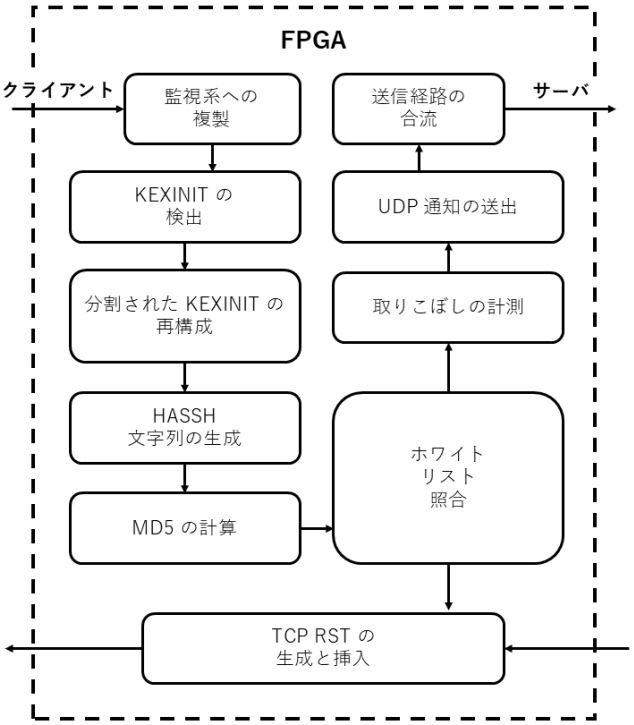


図 2 提案システムの実装構成

す．図 2 の左側の列が解析系，右側の列が送出系であり，最下段の TCP RST の生成と挿入がクライアント側の経路に接続される．以降，図中の各モジュールの処理を上から順に述べる．

4.2.1 監視系へのフレーム複製

本モジュールでは，入力されたフレームを本線用と解析用の 2 つに複製し，それぞれに同じ情報を流す．本線は反対側のポートへ送り出す経路，解析用は HASSH の計算に使う経路であり，解析が遅延しても本線は通常の速度で稼働できる．

解析系に送る際，深さ 256 のバッファを介して複製を送る．1 クロックあたり 64 bit を受け取るため容量は 2,048 byte に相当し，イーサネットフレームの最大長 1,518 byte を 1 フレーム分保持できる．1 フレームを最後まで保存できた際のみ次のモジュールに送り，途中までしか保存できない場合はフレームをそのまま破棄する．後段のモジュールはフレームの先頭から何 byte 目かを数えてヘッダを解釈するため，途中で切れたフレームが届くと位置がずれて誤検出につながる．フレーム単位で破棄することでこれを防いでいる．

4.2.2 KEXINIT の検出

本モジュールは，複製されたフレームを先頭からたどって KEXINIT メッセージを探し，見つけた際に場所を後のモジュールに伝える．フレームの中で，TCP のデータが始まる位置は一定ではない．Internet Protocol version 4 (IPv4) ヘッダの長さは Internet Header Length (IHL) というフィールドで，TCP ヘッダの長さは Data Offset というフィールドで，それぞれ指定される．どちらもクライアントの環境によって変わるため，位置を決め打ちできず，接続の都度求めなければならない．

参照するフィールドとその位置を図 3 に示す．図 3 の上段は，Ethernet ヘッダ，IPv4 ヘッダ，TCP ヘッダ，SSH ペイロードがこの順に並んだフレームの構造を表す．下段の 1 から 7 の番号は，本モジュールが参照するフィールドの位置を先頭からのオフセットで示しており，以下の説明中の番号と対応する．

判定は 2 段階に分かれる．第 1 段では，Ethernet および IPv4 ヘッダ内の固定位置にあるフィールドを確認する．12, 13 byte 目の EtherType が 0x0800 なら IPv4 であり，これが図 3 の 1 に当たる．23 byte 目のプロトコル番号が 6 なら TCP であり，これが 3 に当たる．14 byte 目の IHL が 5 以上であることも確認する．これが 2 に当たる．

第 2 段では，第 1 段で読んだ値を使って位置を計算する．TCP ヘッダは，14 + IHL × 4 byte 目から始まり，これが図 3 の 4 に当たる．そこから 12 byte 目にある Data Offset を読めば，TCP のデータが始まる位置がわかる．前者が 5，後者が 6 に当たる．

SSH のデータは，4 byte の長さ，1 byte のパディング長，1 byte のメッセージ種別，という順で始まる．したがってメッセージ種別は，データ開始位置から 5 byte 目にあり，これが図 3 の 7 に当たる．この値が 0x14 であれば KEXINIT である．

4.2.3 分割された KEXINIT の再構成

KEXINIT メッセージは必ずしも 1 フレームに収まるとは限らず，たとえば OpenSSH の場合は 1,448 byte と 88 byte の 2 つに分かれて届く．そのため本モジュールでは，分割された KEXINIT を 4,096 byte のメモリ上でつなぎ合わせ，全体がそろったものだけ次のモジュールに送る．あわせて，RST を作るために必要な情報である，送信元と宛先の MAC アドレス，IP アドレス，ポート番号，シーケンス番号をレジスタに保持する．

4.2.4 HASSH のもとになる文字列の生成

本モジュールは再構成された KEXINIT を 1 byte ずつ読み，HASSH のもとになる文字列を作る．KEXINIT には 10 本のアルゴリズムリストが並んでおり，HASSH では，このうちクライアント側の鍵交換，暗号，MAC, 圧縮方法を選び，セミコロンでつないだものを使用する．その連結結果を 2,048 byte のバッファに書き出し，次のモジュールに送る．

4.2.5 MD5 の計算

このモジュールでは，元となる文字列の MD5 を計算し HASSH を求める．Request for Comments (RFC) 1321 の手順をそのまま実装した．HASSH の計算は KEXINIT を 1 つ受け取るごとに 1 回行えばよく，通信速度に追従する

© 2026 Information Processing Society of Japan

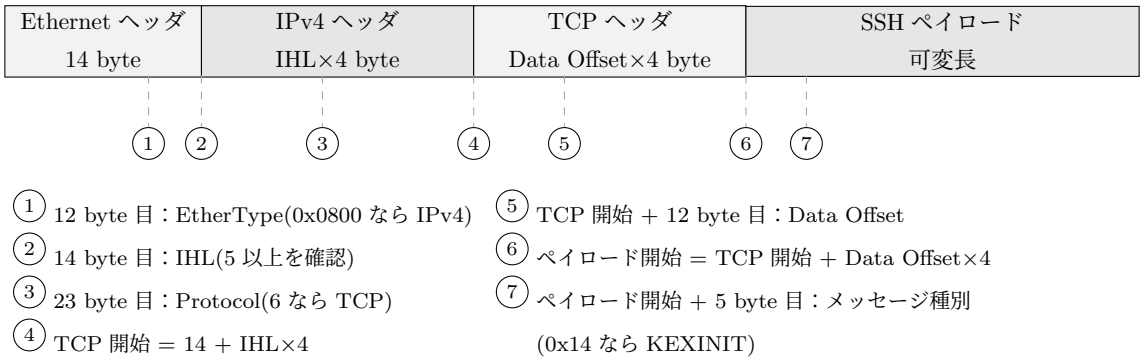


図 3 フレーム構造と参照するオフセット

必要がない．そのため回路規模を優先し，1 ラウンドずつ順に処理する構成とした．

4.2.6 ホワイトリストと照合

本モジュールは，算出された HASSH をホワイトリストと照合し，一致しない場合に警告を出力する．警告には理由コードを付与する．ホワイトリストとの不一致には理由コード 1 を割り当て，解析が完了しなかった場合にはそれ以外の値を割り当てる．遮断の対象となるのは，理由コードが 1 の場合，すなわちホワイトリストとの不一致のみである．ホワイトリストには，実測した OpenSSH 8.9p1 の HASSH である `e1d88fdd485f9c710074daa1fcce80a6` を登録した．

4.2.7 取りこぼしの計測

SSH_MSG_KEXINIT を検出した回数と判定が行われた回数を計上するカウンタを設けた．両者の差が，判定に至らなかった通信の数を表す．ただし本カウンタの読み出し経路は未実装であり，値の取得は今後の課題である．

4.2.8 UDP 通知の送出

本モジュールは，警告を受けるたびに 64 byte の User Datagram Protocol (UDP) フレームを 1 つ送り出す．ペイロードには，送信回数，破棄回数，検出時情報，理由コードを各 1 byte, HASSH を 16 byte, パディングを 2 byte 格納する．

宛先はブロードキャストアドレス 255.255.255.255, 送信元は 192.168.50.254 とし，送信元ポートと宛先ポートはいずれも 5514 とした．宛先をブロードキャストとしたため，IP ヘッダの全フィールドが定数となる．そのため IP チェックサムもあらかじめ算出した値 0x8715 を埋め込んでおり，実行時には計算していない．

フレーム長を 64 byte としたのは，64 bit 幅のバスでちょうど 8 クロックに収まり，最終クロックの有効 byte 数を求める回路と MAC 層による自動パディングへの依存を避けられるためである．

4.2.9 送信経路の合流

本モジュールは，本線を転送するストリームと UDP 通知を送出するストリームを，1 つの送信経路にまとめる．イーサネットフレームは途中で他のフレームに切り替えることができないため，一方のフレームを送出している間は，もう一方を待たせる必要がある．本モジュールはフレームの先頭を受け付けた時点で送出元を固定し，末尾まで送り終えてから切り替える．UDP 通知は 64 byte, すなわち 8 クロック分であり，この間だけ本線側が待たされる．

4.2.10 TCP RST の生成と挿入

本モジュールは，遮断の指示を受けたときに TCP RST を 1 つ作り，サーバからクライアントへ向かう経路に差し込む．

本装置は経路上に位置するためフレームを破棄して遮断することもできるが，RST の注入を選んだ．破棄による遮断では本線に遮断の判断を反映する経路が必要となり，解析側の異常が通信の停止につながる一方，RST の注入は本線を変更せずフレームを 1 つ追加する操作であるため，この問題が生じない．また判定が完了する時点で SSH_MSG_KEXINIT の転送はすでに終わっており，フレーム自体を破棄するには転送の保留や接続を記憶するテーブルが必要になる．RST を用いれば，受け取ったクライアントが接続を終了し，サーバが次にフレームを送出した時点でサーバ側も解放される．RST の各フィールドは，サーバが送ったように見せかけるため，KEXINIT を再構成する際に抜き取った値の送信元と宛先を入れ替えて使う．生成するフレームは 60 byte とし，送信元ポートを 22 番，宛先ポートを観測したクライアントの送信元ポートに設定する．TCP フラグには RST および Acknowledgment (ACK) を指定し，シーケンス番号には観測した ACK 番号を用いる．これはサーバが次に送る番号であり，クライアントが期待している番号でもある．

生成した RST は，通常フレームの転送完了を待ってフレーム境界で挿入する．RST 送出時のみ生成フレームへ切り替え，通常時は受信したフレームをそのまま転送する．

5. 提案システムの検証

本章では，提案システムの検証方法について述べる．検証では，ホワイトリストに掲載したクライアントと掲載し

© 2026 Information Processing Society of Japan

表 1　検証環境

	クライアント	サーバ
OS	Ubuntu 22.04.5 LTS	Ubuntu 24.04.4 LTS
カーネル	6.8.0	6.8.0
CPU	Core i3-14100	Core i5-12400
メモリ	8 GB (DDR5)	16 GB (DDR5)
NIC	Intel 82599ES	Intel 82599ES
SSH	OpenSSH 8.9p1	OpenSSH 9.2p1
役割	攻撃・負荷生成	監視対象

ていないクライアントの双方から接続を行い，遮断動作と掲載クライアントへの影響を確認するとともに，算出された HASSH の正しさと遮断が成立するまでの時間を測定する．

5.1　システム環境

検証環境を表 1 に示す．表 1 には，クライアントとサーバのそれぞれについて，Operating System (OS)，カーネルのバージョン，CPU，メモリ，Network Interface Card (NIC)，SSH の実装および役割を示す．クライアントとサーバの 2 台の Personal Computer (PC) を FPGA を挟んで接続し，その両側のインタフェースでパケットキャプチャを取得した．両 PC の NIC は Intel 82599ES であり，10GBase-SR で接続した．攻撃には Hydra を用い，これは SSH 接続に libssh 0.9.6 を使用する．背景トラフィックの生成には iperf3 を用いた．

FPGA は Intel Cyclone 10 GX 開発キット上に構築した．論理合成には Quartus Prime Pro 20.1 を，高位合成には High-Level Synthesis (HLS) Compiler Pro 20.1 を用いた．データパスの動作周波数は 156.25 MHz である．

5.2　遮断動作の確認方法

FPGA をクライアント PC と SSH サーバの間に挿入し，クライアント側とサーバ側の両方のインタフェースでパケットキャプチャを取得する．提案システムは通知をサーバ側の経路に，RST をクライアント側の経路に送出するため，両者は別々のインタフェースで観測される．サーバは 192.168.50.20, クライアントは 192.168.50.10 とし，ホワイトリストには OpenSSH 8.9p1 の HASSH を 1 件だけ登録する．

この状態で，次の 2 通りの接続を行う．

- 1 つ目は，ホワイトリストに掲載した OpenSSH 8.9p1 による接続である．
- 2 つ目は，ホワイトリストに掲載していない攻撃ツール Hydra による接続である．

前者では鍵交換と認証が最後まで進み，通知および RST が送出されないことを確認する．後者では通知と RST が送出され，認証に到達する前に接続が切断されることを確認する．

5.3　HASSH の算出結果の確認方法

UDP 通知として送出された HASSH を，同一の SSH_MSG_KEXINIT からソフトウェア上で算出した値と比較する．

5.4　高負荷時の検出の確認方法

監視対象外の通信で回線を占有した状態でも攻撃を検出できるかを確認する．iperf3 を用いてサーバのポート 5201 宛に UDP トラフィックを送出し，その状態で Hydra による接続を 5 回試行する．

5.5　遮断までの時間の測定方法

tcpdump で取得したキャプチャの時刻を用い，2 種類の時間を測定する．1 つはサーバ側において，SSH_MSG_KEXINIT が到着してから UDP 通知が到着するまでの時間である．もう 1 つはクライアント側において，SSH_MSG_KEXINIT を送出してから TCP RST が到着するまでの時間である．いずれも同一インタフェース内の時刻差から求める．2 つのキャプチャの間には計測系に起因する時刻のずれがあるため，インタフェースをまたいだ比較は行わない．

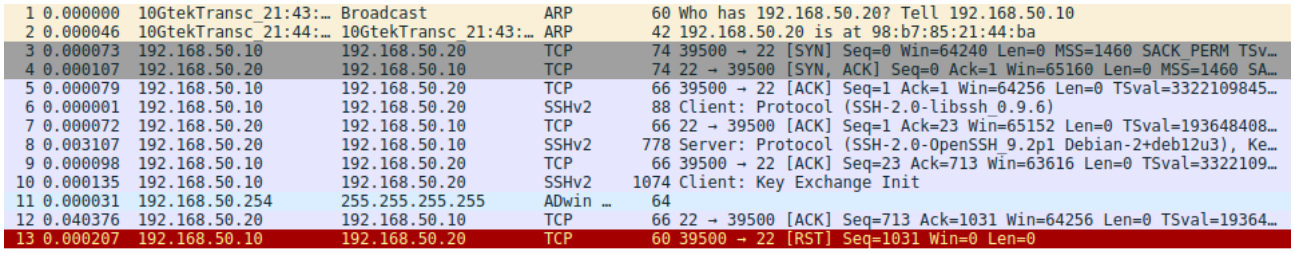
6.　評価結果

6.1　遮断動作の結果

提案システムを通過した通信のパケットキャプチャを図 4 に示す．図 4 は，各行が 1 つのフレームに対応しており，左から順に通番，先頭フレームからの経過時間，送信元アドレス，宛先アドレス，プロトコル，フレーム長および概要を示す．はじめに，ホワイトリストに掲載した OpenSSH 8.9p1 で接続した．鍵交換，新規鍵の交換，認証が順に進行し，通知および RST は送出されなかった．提案システムは理由にかかわらず警告時には通知を送出するため，通知が 1 件も観測されなかったことは，警告そのものが発生しなかったことを示す．

次に，ホワイトリストに掲載していない Hydra(SSH 接続に libssh 0.9.6 を使用) で接続した．この接続における SSH_MSG_KEXINIT の送出, UDP 通知の送出および TCP RST の注入に対応する部分を図 4 に示す．クライアントが SSH_MSG_KEXINIT を送出した直後に，サーバ側ではブロードキャスト宛の UDP 通知が，クライアント側ではサーバの送信元ポート 22 からクライアント宛の RST が観測された．注入された RST のシーケンス番号はサーバが次に送出する番号と，確認応答番号は SSH_MSG_KEXINIT の末尾と一致しており，クライアントはこれを受理して接続を終了した．鍵交換は完了しておらず，認証には到達していない．

© 2026 Information Processing Society of Japan



1	0.000000	10GtekTransc_21:43:…	Broadcast	ARP	60 Who has 192.168.50.20? Tell 192.168.50.10
2	0.000046	10GtekTransc_21:44:…	10GtekTransc_21:43:…	ARP	42 192.168.50.20 is at 98:b7:85:21:44:ba
3	0.000073	192.168.50.10	192.168.50.20	TCP	74 39500 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSv…
4	0.000107	192.168.50.20	192.168.50.10	TCP	74 22 → 39500 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SA…
5	0.000079	192.168.50.10	192.168.50.20	TCP	66 39500 → 22 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=3322109845…
6	0.000001	192.168.50.10	192.168.50.20	SSHv2	88 Client: Protocol (SSH-2.0-libssh_0.9.6)
7	0.000072	192.168.50.20	192.168.50.10	TCP	66 22 → 39500 [ACK] Seq=1 Ack=23 Win=65152 Len=0 TSval=193648408…
8	0.003107	192.168.50.20	192.168.50.10	SSHv2	778 Server: Protocol (SSH-2.0-OpenSSH_9.2p1 Debian-2+deb12u3), Ke…
9	0.000098	192.168.50.10	192.168.50.20	TCP	66 39500 → 22 [ACK] Seq=23 Ack=713 Win=63616 Len=0 TSval=3322109…
10	0.000135	192.168.50.10	192.168.50.20	SSHv2	1074 Client: Key Exchange Init
11	0.000031	192.168.50.254	255.255.255.255	ADwin …	64
12	0.040376	192.168.50.20	192.168.50.10	TCP	66 22 → 39500 [ACK] Seq=713 Ack=1031 Win=64256 Len=0 TSval=19364…
13	0.000207	192.168.50.10	192.168.50.20	TCP	60 39500 → 22 [RST] Seq=1031 Win=0 Len=0

図 4　提案システムを通過した際のパケットキャプチャ

```
0x0000:  4500 0032 0000 0000 4011 8715 c0a8 32fe  E..2....@.....2.
0x0010:  ffff ffff 158a 158a 001e 0000 0000 0001  ................
0x0020:  3335 afa7 a75e 84f8 348f 05c6 23c5 ecf9  35...^..4...#...
0x0030:  0000                                     ..
```

図 5　送出された UDP 通知フレーム

6.2　HASSH の算出結果

送出された UDP 通知フレームを図 5 に示す．図 5 は通知フレームを 16 進数で表示したものであり，左端の数値が先頭からのオフセットを，0x0022 以降がペイロードを表す．

このフレームでは，IP ヘッダの検査和が設計で埋め込んだ定数 0x8715 と一致し，ペイロード先頭から 4 byte 目の理由コードは 1，すなわちホワイトリスト不一致であった．また，ペイロード先頭から 2 byte 目の破棄回数は 0 であり，通知そのものの取りこぼしは発生していない．なお，SSH_MSG_KEXINIT の検出漏れを数えるカウンタは読み出し経路が未実装であり，検出漏れの有無は本測定では確認していない．

Hydra の接続に対して FPGA が算出した HASSH は `3335afa7a75e84f8348f05c623c5ecf9` であった．同一の SSH_MSG_KEXINIT からソフトウェア上で算出した値もこれと一致した．

OpenSSH 8.9p1 については，一致した場合に通知が送出されないため FPGA 内部の値を直接読み出してはいないが，登録した値 `e1d88fdd485f9c710074daa1fcce80a6` と一致したことから，同じ値が算出されたと判断できる．

以上から，FPGA 上で SSH_MSG_KEXINIT を解析し，HASSH を正しく算出できることを確認した．

6.3　遮断までの時間

サーバ側では，SSH_MSG_KEXINIT の到着から UDP 通知の到着までが 31 μs であった．いずれも同一インタフェースの受信時刻であり伝送遅延が相殺されるため，この値は FPGA 内部の解析時間に相当する．

クライアント側では，SSH_MSG_KEXINIT の送出から TCP RST の到着までが 57 μs であった．この値にはクライアントの NIC およびキャプチャ処理に要する時間が含まれるため，遮断が成立するまでの時間の上限にあたる．

SSH では鍵交換が完了したのちに認証が行われる．本測定では鍵交換の途中で接続が切断されており，認証試行の前に遮断が成立したことを示している．

6.4　高負荷時の検出

iperf3 により平均 4.17 Gbps の UDP トラフィックを 30 秒間送出した際の，受信側での損失率は 0.013%であった．同時に Hydra による接続を 5 回試行したところ，いずれも UDP 通知が送出され，5 回すべてを検出した．通知フレーム内の破棄回数もすべて 0 であり，通知の取りこぼしも発生していない．監視対象外の通信で回線を占有した状態でも，攻撃の検出が維持されることを確認した．

7.　考察

本章では，評価結果をもとに提案システムの特徴と限界について述べる．提案システムは鍵交換の最初のメッセージである SSH_MSG_KEXINIT を対象とするため，認証よりも前に判定できる．遮断までの 57 μs は，鍵の切り替え完了までに要した 20.5 ms より 2 桁以上短く，認証試行の前に十分間に合う．

7.1　ホワイトリスト方式の利点と限界

ブラックリスト方式では，登録されていない未知の攻撃ツールに対応できない．これに対しホワイトリスト方式では，登録されていないクライアントはすべて遮断の対象となるため，未知のツールであっても遮断できる．

また，本線は組合せ回路による直結でレジスタ段を持たないため解析の負荷を受けず，装置の挿入による往復遅延の増分は，直結時と比べて中央値で 1 μs 以下であった．

一方で，正規の利用者であっても未登録のクライアントを使用した場合には遮断される．そのため，利用するクライアントを事前に把握できる環境が前提となる．不特定多数に公開するサーバには適さない．

また，HASSH は実装ごとの指紋であり，個体を識別するものではない．攻撃者がホワイトリストに登録されているものと同じ実装，たとえば標準の OpenSSH をそのまま使用した場合には，同じ HASSH が算出されるため遮断できない．この場合は，従来の認証失敗回数に基づく対策と併用する必要がある．

SSH ブルートフォース攻撃はパスワード認証を対象とす

© 2026 Information Processing Society of Japan

るため，サーバの設定を変更できる環境では，パスワード認証の無効化が最も確実である．提案方式が意味を持つのは，機器の仕様や既存システムとの互換性からパスワード認証を無効にできない場合と，複数のサーバを保護する場合である．提案システムは経路上に配置するため，各サーバの設定を変更せずに配下のサーバをまとめて対象にできる．

7.2 解析できない通信の扱い

提案システムは，解析できなかった通信を遮断せずそのまま転送する．遮断の指示は理由がホワイトリスト不一致である場合にのみ生成されるため，ヘッダ長が標準的でない場合や SSH_MSG_KEXINIT が 3 つ以上に分割された場合には，遮断を指示する信号そのものが生成されない．誤った遮断は正常な通信を壊すのに対し見逃しはカウンタに残るため，可用性を優先した設計としている．

7.3 回避可能性

提案方式は攻撃者が装置の存在を知らない場合を想定している．装置の動作を把握した攻撃者に対しては，次の回避手段が成立する．1 つ目は，HASSH の詐称である．HASSH の算出に用いるアルゴリズムリストは認証されないため，正規クライアントと同じリストを送れば通過する．2 つ目は，解析できない形式の使用であり，IP オプションの付加や TCP セグメントの細分化により解析の対象から外れる．3 つ目は，意図的な高負荷であり，監視系のバッファが不足すればフレーム単位で破棄される．ただし本測定では，平均 4.17 Gbps の背景トラフィック下でも検出は維持された．4 つ目は，RST の無視であり，カーネルの TCP 処理を経由しない実装では接続を継続できる．これらのうち 1 つ目は方式に由来し，2 つ目から 4 つ目は実装に由来する．したがって提案方式は単独で攻撃を防ぐものではなく，既知の攻撃ツールをそのまま用いる接続を排除する層として位置づける．

7.4 運用上の制約

ホワイトリストは合成時のパラメータとして回路に埋め込まれている．クライアントを追加するには値を書き換えて論理合成をやり直す必要があり，数十分を要する．運用を考えると，動作中に書き換えられる経路を設けることが望ましい．

8. 結論

本研究では，SSH_MSG_KEXINIT から算出した HASSH を用いて SSH クライアントを識別し，ホワイトリストに掲載されていないクライアントからの接続を TCP RST の注入によって遮断する NIPS を提案し，FPGA 上に実装した．実装は，フレームの複製から TCP RST の生成と挿入までの 8 つのモジュールから構成される．これらをすべて FPGA 上に実装することで，ホストへの問い合わせを介さずに遮断を完結させた．実機による確認では，ホワイトリストに掲載した OpenSSH 8.9p1 の接続は最後まで通過し，掲載していない攻撃ツール Hydra の接続は認証に到達する前に切断された．算出された HASSH はソフトウェア上で算出した値と一致した．また，FPGA 内部の解析時間は 31 μs, SSH_MSG_KEXINIT の送出から遮断が成立するまでの時間は 57 μs 以下であり，鍵交換に要する時間と比べて大幅に短かった．

今後の課題は 3 点である．1 つ目は，10 Gbps の入力に対する測定であり，トラフィック生成側の増強と FPGA 内部のカウンタを読み出す経路の実装が必要である．2 つ目は，ホワイトリストを動作中に書き換える機構の実装である．3 つ目は，同一実装を用いる攻撃への対応であり，認証失敗回数に基づく従来手法との併用を検討する．

謝辞

本研究の一部は，JSPS 科研費 23K28086 の支援により行った．

参考文献

[1] 国立研究開発法人情報通信研究機構, “NICTER 観測レポート 2025,” https://csl.nict.go.jp/report/NICTER_report_2025.pdf (Accessed 2026-7-13).

[2] Salesforce, “HASSH,” https://github.com/salesforce/hassh (Accessed 2026-8-16).

[3] Z. Zhao, H. Sadok, N. Atre, J.C. Hoe, V. Sekar and J. Sherry, “Achieving 100Gbps Intrusion Prevention on a Single Server,” Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI), pp. 1083-1100, 2020.

[4] K. Jaic, M.C. Smith and N. Sarma, “A Practical Network Intrusion Detection System for Inline FPGAs on 10GbE Network Adapters,” Proceedings of the 25th IEEE International Conference on Application-Specific Systems, Architectures and Processors (ASAP), pp. 180-181, 2014.

[5] M. Javed and V. Paxson, “Detecting Stealthy, Distributed SSH Brute-Forcing,” Proceedings of the 20th ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 85-96, 2013.

[6] V. Ghiëtte, H. Griffioen and C. Doerr, “Fingerprinting Tooling Used for SSH Compromisation Attempts,” Proceedings of the 22nd International Symposium on Research in Attacks, Intrusions, and Defenses (RAID), pp. 61-71, 2019.

[7] F. Baiardi, F. Boni, G. Braccini, E. Briganti and L. Deri, “Kex-Filtering: A Proactive Approach to Filtering,” Proceedings of the 21st International Conference on Security and Cryptography (SECRYPT), pp. 528-535, 2024.

[8] homelith, “c10gx_icmp_server,” https://github.com/homelith/c10gx_icmp_server (Accessed 2026-7-13).

© 2026 Information Processing Society of Japan